

Assignment 2

Calum Murphy

2025-04-11

Install necessary libraries

```
library(quantmod)

## Loading required package: xts
## Loading required package: zoo
##
## Attaching package: 'zoo'
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
## Loading required package: TTR
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
library(gramEvol)
library(glue)
```

Background to the problem

Stock price prediction is a fundamental challenge in quantitative finance and algorithmic trading and the ability to accurately predict a stock's share price can guide investment decisions, maximise profit and minimise risk. However, predicting the future is never easy, and those who buy and sell stocks should know that.

Various models have been employed by algorithmic traders to try and accurately determine future stock prices such as linear regression models or neural networks. This report focuses on a Grammatical Evolution (GE) approach to forecasting future stock price. Grammatical Evolution is a search algorithm which uses techniques based on the process of evolution found in the real world to determine a relationship between data i.e. a formula. This approach is also called a symbolic regression and has been successful in determining Kepler's third law of planetary motion (Ephorie.de. (2019)), which in reality took decades to develop.

This approach has been used in the past to predict stock price (e.g. D'Mello, L., Jeswani, A. and Johnson, J. (2020)) and we will utilise it here for this report.

We will use Netflix historical stock price data from a 6-month period (2024-10-01 to 2025-04-01) for this report. After we have found the best expression for the data and used it to generate our predicted prices, we will then use a simple trading rule to inform us if we should buy, sell or hold the stock. We will then evaluate our profit from our forecasted values against some unseen data to test how well our model is performing.

Once we have completed all of the above, we will test our model against a simple linear regression to compare results.

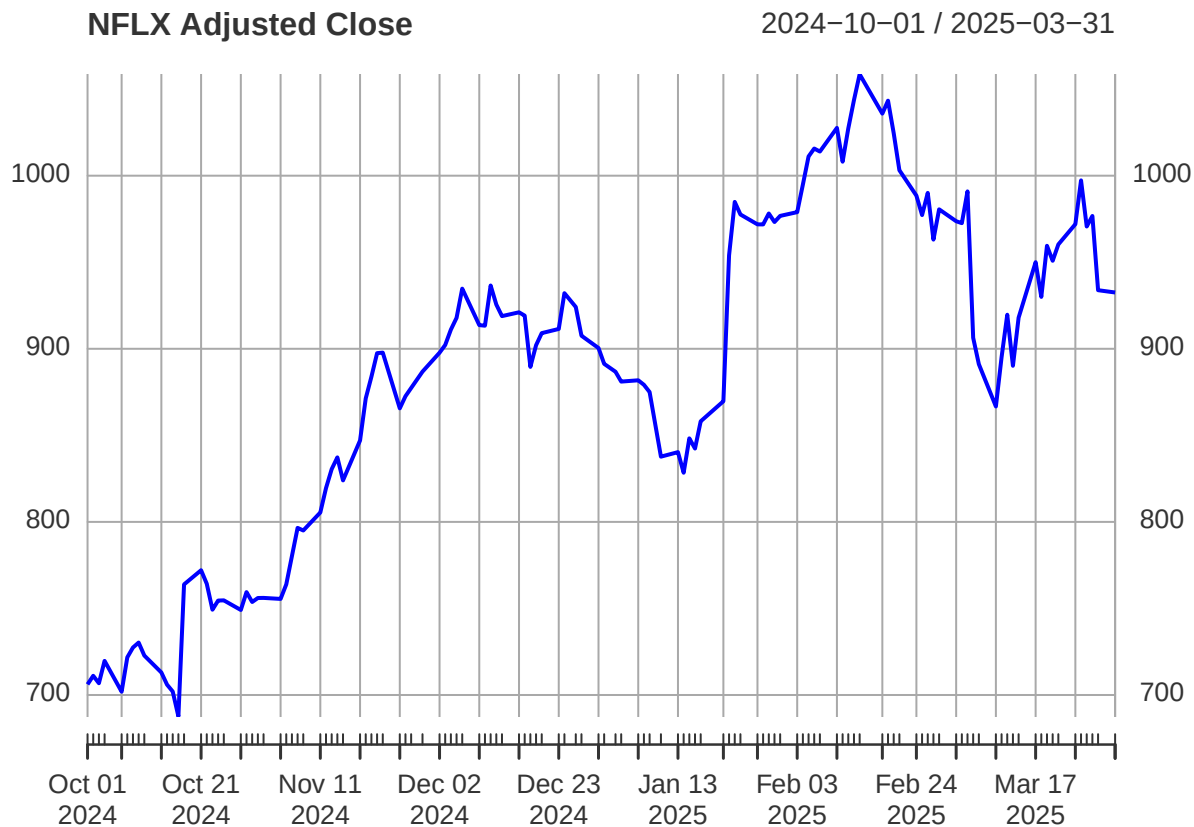
Load stock data

```
stock <- c("NFLX")
getSymbols(stock, src="yahoo", from="2024-10-01", to="2025-04-01")

## [1] "NFLX"

netflix_data <- coredata(NFLX)
netflix_data<-data.frame(date=index(NFLX)
                        , coredata(NFLX))

plot(NFLX$NFLX.Adjusted, main="NFLX Adjusted Close", col="blue")
```



Standard deviation of NFLX Adjusted Close Price

```
glue('Standard deviation of NFLX Adjusted Close Price: {round(sd(netflix_data$NFLX.Adjusted),2)}')
```

```
## Standard deviation of NFLX Adjusted Close Price: 96.54
```

A fairly volatile stock.

Feature selection

I experimented with different features such as EMA, SMA, 3-day lag, 4-day lag but I found the best set of features was a 5-day lag including only the previous days stock prices. This was chosen because it had the lowest forecast RMSE (see below). If I had more time I would like to have implemented a GA to optimise feature selection rather than having to trial and error myself.

```

#This data frame holds the previous 5 day stock prices
myNFLXdata <- data.frame(
  x5 = as.numeric(Lag(NFLX$NFLX.Adjusted, 5)),
  x4 = as.numeric(Lag(NFLX$NFLX.Adjusted, 4)),
  x3 = as.numeric(Lag(NFLX$NFLX.Adjusted, 3)),
  x2 = as.numeric(Lag(NFLX$NFLX.Adjusted, 2)),
  x1 = as.numeric(Lag(NFLX$NFLX.Adjusted, 1)),
  x  = as.numeric(NFLX$NFLX.Adjusted)
)

#Removes NaN values
myNFLXdata <- na.omit(myNFLXdata)

traindata <- myNFLXdata[1:(nrow(myNFLXdata)-5), ] #Train data
testdata <- myNFLXdata[(nrow(myNFLXdata)-4):nrow(myNFLXdata), ] #The unseen next week of trades we will

```

Implementing GE

This is a standard method for all GE models. The best expression aims to minimise the root mean squared error, RMSE (defined in the fitness function), as this is a very popular metric to evaluate forecasted stock price.

```

newRules <- list(
  expr = grule(op(expr, expr), func(expr), var),
  func = grule(sin, cos, log, exp),
  op = grule('+', '-', '*', '/'),
  var = grule(mydata$x5, mydata$x4, mydata$x3, mydata$x2, mydata$x1)
)

newGram <- CreateGrammar(newRules)

# Fitness function using RMSE
newFitFunc <- function(expr) {
  result <- eval(expr)
  if (any(is.nan(result))) return(Inf)
  return(sqrt(mean((mydata$x - result)^2)))
}

```

Train GE on training data

```

# View best evolved expression based on training data
ge

## Grammatical Evolution Search Results:
##   No. Generations: 1000
##   Best Expression: mydata$x1 - cos(mydata$x2)
##   Best Cost:      19.6123139079527

ge$best$expressions

```

```
## expression(mydata$x1 - cos(mydata$x2))
```

We will now use this expression and test it on our training and testing sets.

```
# Evaluate performance on training data
train_predictions <- eval(ge$best$expressions)
train_rmse <- sqrt(mean((traindata$x - train_predictions)^2))
```

```
# Evaluate performance on test data
mydata <- testdata
test_predictions <- eval(ge$best$expressions)
test_rmse <- sqrt(mean((testdata$x - test_predictions)^2))
```

```
glue("Train RMSE: {train_rmse}")
```

```
## Train RMSE: 19.6123139079527
```

```
glue("Test RMSE:{test_rmse}")
```

```
## Test RMSE:25.90854272734
```

Due to the magnitude of the stock price, both RMSE scores are respectable, indicating a good evolved expression.

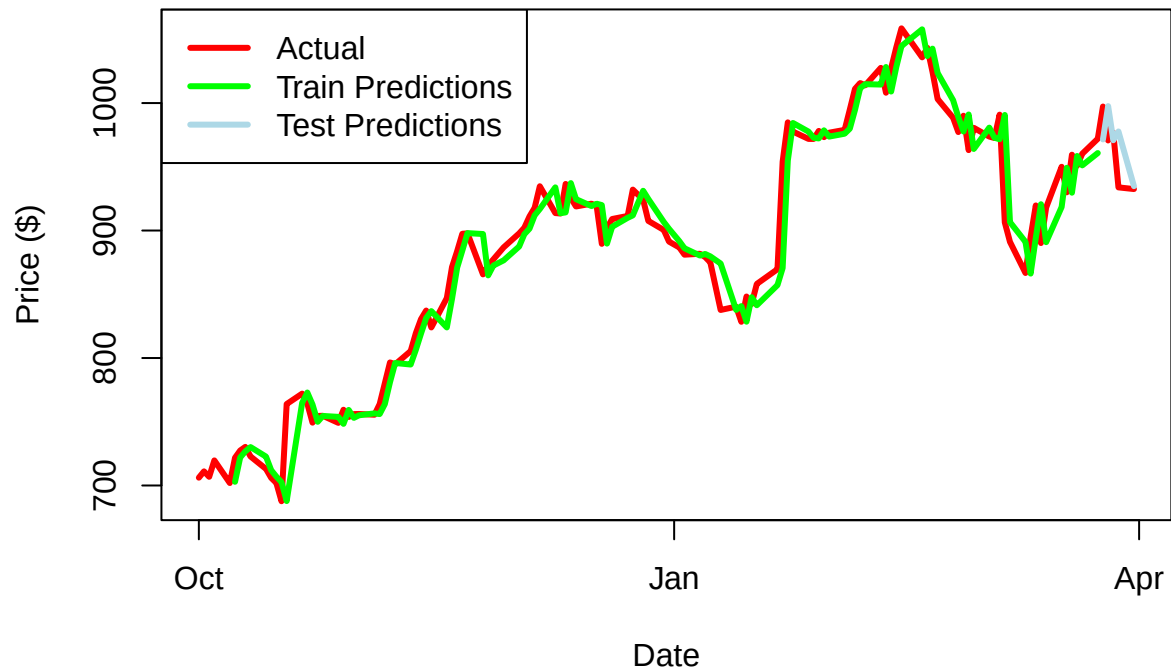
Visualising train and test predictions

```
plot(netflix_data$date, netflix_data$NFLX.Adjusted, type="l", col="red", lwd=3, xlab="Date", ylab='Price')
lines(netflix_data[6:(nrow(mynflxdata)),]$date, train_predictions, type="l", col="green", lwd=3)

lines(tail(netflix_data$date, 5), test_predictions, type="l", col="lightblue", lwd=3)

legend("topleft", legend = c("Actual", "Train Predictions", "Test Predictions"), col = c("red", "green", "lightblue"))
```

NFLX Real vs Evolved stock price



#Because of lagged values, training begins on day 5

Predicting future stock price

We will use the best evolved expression to forecast the values for the next week i.e next 5 days.

Forecast next 5 values using symbolic regression

```
mydata <- testdata[1, 1:5] # first row of test data
```

```
predictions <- c()
```

#This for loop shifts each lag bringing the previous prediction to be used by the GE for the next day

```
for (i in 1:nrow(testdata)) {
  predictions[i] <- eval(ge$best$expressions)
  # shift lags

  mydata[5] <- mydata[4]
  mydata[4] <- mydata[3]

  mydata[3] <- mydata[2]
  mydata[2] <- mydata[1]
  mydata[1] <- predictions[i]
}
```

Compare predictions with test data (Back-testing)

```

actuals <- testdata$x
predictions

## [1] 971.4815 960.7766 951.1034 958.4923 930.7256

forecast_rmse <- sqrt(mean((actuals - predictions)^2))

glue("Forecast RMSE: {round(forecast_rmse,4)}")

```

Forecast RMSE: 20.1482

A respectable RMSE score considering the volatility and range of the dataset.

Visualising the comparison of predicted values against real values

```

#head(testdata,1) - Genrates first row of test data and index

glue("Date we're prediciting: {netflix_data[120,]$date}")

```

Date we're prediciting: 2025-03-25

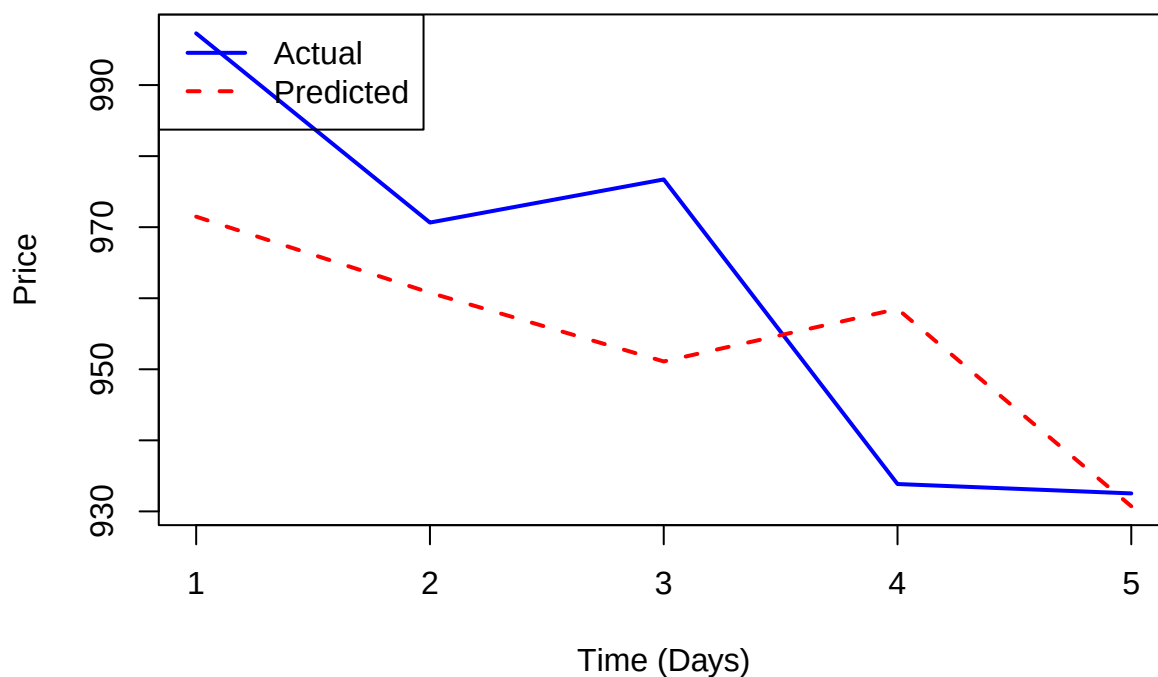
```

plot(actuals, type = "l", col = "blue", lwd = 2, ylim = range(c(actuals, predictions)), xlab = "Time (Days)",
lines(predictions, type = "l", col = "red", lwd = 2, lty = 2)

legend("topleft", legend = c("Actual", "Predicted"), col = c("blue", "red"),
      lty = c(1, 2), lwd = 2)

```

Predicted vs Actual Prices



Here we can see that while not perfect, the best expression is able to forecast the downward trend of the stock data and converges to a very accurate prediction on day 5 (31st March). Initially I tried increasing the

test set, however, the forecasted predictions on the increased test set were not accurate over the long term. This suggests the need for constant training and evaluation on short term data, rather than applying the best expression to a data set which would undoubtedly experience different trends (e.g. New tariffs brought in, pandemic etc.)

Using predicted prices to develop trading rule and evaluate profit

We will develop a simple trading rule and will evaluate our profit based on that rule by testing our strategy on the test set.

```
#Let's say we currently own 15 NFLX shares

number_of_shares = 15

#Current stock price
current_price = tail(traindata$x, 1)

current_holdings = number_of_shares * current_price

glue("Current holdings: £{round(current_holdings,2)}")

## Current holdings: £14579.85

#Expected next day price
expected_next_day_price = eval(ge$best$expressions)

expected_holdings = number_of_shares * expected_next_day_price

glue("Expected holdings after one day: £{round(expected_holdings,2)}")

## Expected holdings after one day: £14559.44

#Actual next day price
actual_price = actuals[1]

actual_holdings = number_of_shares * actual_price

glue("Actual holdings after one day: £{round(actual_holdings,2)}")

## Actual holdings after one day: £14959.2
```

Stocks can experience fluctuations from day to day so it is better to see it smoothed out over a week (5 days) and will develop a trading rule to indicate if we should buy more or sell. We shall say that if the price of stock experiences a drop of 5 dollars we will sell. If it predicts an increase of 5, we will buy 5 shares. Otherwise we will hold.

This is a very simple, and risk averse, trading rule that will help us illustrate how well our model is performing.

Trading rule on future one week price

```
glue("Current holdings: £{round(current_holdings,2)}")

## Current holdings: £14579.85

#Expected price in one week
expected_one_week_price = predictions[5]

expected_holdings_one_week = number_of_shares * expected_one_week_price
```

```

glue("Expected holdings after one week: £{round(expected_holdings_one_week,2)}")

## Expected holdings after one week: £13960.88
#Actual one week price
actual_one_week_price = actuals[5]

actual_one_week_holdings = number_of_shares * actual_one_week_price

glue("Actual holdings after one week: £{round(actual_one_week_holdings,2)}")

## Actual holdings after one week: £13987.95
if ((current_price - predictions[5]) > 10) {
  action = "SELL"
  print(action)
} else if ((current_price - predictions[5]) < -10) {
  print("BUY")
  print(action)
} else {
  action = "HOLD"
  print(action)
}

## [1] "SELL"

```

With our trading rule telling us to sell, we are now able to evaluate profit.

Evaluate profit

```

expected_profit = expected_holdings_one_week - current_holdings

actual_profit = actual_one_week_holdings - current_holdings

if (action == "SELL"){
  expected_profit = abs(expected_profit)
  actual_profit = abs(actual_profit)
}

glue("Expected profit after one week: £{round(expected_profit,2)}")

## Expected profit after one week: £618.97
glue("Actual profit after one week: £{round(actual_profit,2)}")

## Actual profit after one week: £591.9

```

The expected profit is slightly larger than our actual profit but not by much. If this model was used to guide investment decisions for a client then I think they would be very pleased with this. Our current holding of shares is quite small, but with a large fund, this model could be used to generate significant gains by forecasting future stock prices.

To finish, we will compare the effectiveness of this model against a simple linear regression.

Comparison of GE vs Linear Regression

```
lm_model <- lm(x ~ x1 + x2 + x3 + x4 + x5, data = traindata)
summary(lm_model)

##
## Call:
## lm(formula = x ~ x1 + x2 + x3 + x4 + x5, data = traindata)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -82.324 -10.786  -0.208   9.904  81.554
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 34.49111    17.81602   1.936  0.0555 .
## x1           0.98022     0.09547  10.268 <2e-16 ***
## x2           0.03554     0.13344   0.266  0.7905
## x3          -0.06077     0.13334  -0.456  0.6495
## x4          -0.05664     0.13453  -0.421  0.6746
## x5           0.06543     0.09515   0.688  0.4931
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 19.76 on 108 degrees of freedom
## Multiple R-squared:  0.9559, Adjusted R-squared:  0.9539
## F-statistic: 468.3 on 5 and 108 DF,  p-value: < 2.2e-16

# Training predictions and RMSE
lm_train_preds <- predict(lm_model, newdata = traindata)
lm_train_rmse <- sqrt(mean((traindata$x - lm_train_preds)^2))

# Test predictions and RMSE
lm_test_preds <- predict(lm_model, newdata = testdata)
lm_test_rmse <- sqrt(mean((testdata$x - lm_test_preds)^2))

glue("Symbolic Regression Train RMSE: {train_rmse} \n")

## Symbolic Regression Train RMSE: 19.6123139079527
glue("Linear Regression Train RMSE: {lm_train_rmse} \n\n")

## Linear Regression Train RMSE: 19.2308310408077
glue("Symbolic Regression Test RMSE: {test_rmse} \n")

## Symbolic Regression Test RMSE: 25.90854272734
glue("Linear Regression Test RMSE: {lm_test_rmse} \n\n")

## Linear Regression Test RMSE: 24.7459445851002
forecast_lm_rmse <- sqrt(mean((actuals - lm_test_preds)^2))
glue("Forecast Symbolic Regression RMSE: {forecast_rmse} \n")

## Forecast Symbolic Regression RMSE: 20.1482317007394
```

```
glue("Forecast Linear Regression RMSE: {forecast_lm_rmse} \n")
```

```
## Forecast Linear Regression RMSE: 24.7459445851002
```

Here we can see that against a simple linear regression, the symbolic regression performs better in almost all metrics. The forecast RMSE (the most important metric) of the GE performs better than the linear model, which informs us the GE is better suited to forecast future stock prices.

Predicting the price of stock is a very hard thing to do. It cannot be boiled down to a simple mathematical expression and must take into account the news and events of the world that may influence the price of a stock. While the algorithm has performed well on this test set, it may not perform as well on future data, which tells us that the model must be constantly retrained on updated historical training data. The algorithm may tell us what it thinks the predicted price will be, but it should be a final human decision on whether to go through with the trade or lack thereof based on an informed view of the current state of affairs.

References

- [1] Ephorie.de. (2019). Symbolic Regression, Genetic Programming... or if Kepler had R – Learning Machines. [online] Available at: <https://blog.ephorie.de/symbolic-regression-genetic-programming-or-if-kepler-had-r> [Accessed 11 Apr. 2025].
- [2] D'Mello, L., Jeswani, A. and Johnson, J. (2020). Stock Price Prediction Using Grammatical Evolution. Algorithms for Intelligent Systems, pp.379–389. doi:https://doi.org/10.1007/978-981-15-3242-9_36.